

Football Data Analysis with YOLO Models

Our Project Repository: https://github.com/IUmutAydin/Football_Data_Analysis

- **Scope of Our Project and Goals with It**

Firstly, football may be the most popular sport in Europe but in general, we take it into account as entertainment as expected from general tracker of a sport branch. If most of football-lovers could contribute to sport analytic at least in a starter manner, this could be super improvement for sport analytics.

We aim to provide some of the basic core data observing with predefined models like YOLO V8 object detection and pose estimation in this project. Due to context of our effort, which is about creating a demo, we have defined some beneficial boundaries on football data categories like distance, speed of players, ball territory of teams etc. It is mandatory to remember that if anyone demands to get related data listed above, some detection, classification and homograph models must be implemented but in according to time consumption of implementing these models from scratch, it is impossible to implement and train these models. For example, if you cannot find exact position of player then you cannot obtain precious results of player movement and it probably causes the wrong analysis. We will dive deeper into these three models but before it, we must clarify what we aimed at with the project. In developing football data analytic field, we have had to get familiar with detection models and some mini-transformer models so that we manage to get underlying data such as player position, team classification and ball territory. We assumed that after implementing the project, we were going to move to more complex concepts like re-identification, action spotting etc. Lastly, we must underly again that we don't implement these YOLO models and we cannot compare two different models although we are enabled to compare our modified project's features with baseline repositories which will be referred to in this documentation.

- **Why We Have Chosen Yolo Models for Detection and Pose Estimation**

We decided to choose YOLOv8 detection and pose estimation models as base models which were going to be fine-tuned. Well-documented and easy to implement structure of YOLO models were the primary reason for our decision. We got the pre-trained weights of these models, and we applied some fine-tune processes with large datasets.

- **YOLO V8 Detection Model Introduction**

We had to detect player positions and luckily, we are rewarded with state-of-the-art model YOLO. We will not explain whole architecture of YOLO models, at least in this documentation, but we will state all the fine-tune steps and usage of models in our project. Thanks to capabilities of YOLO v8 detection model we got player bounding boxes and their respective locations in frames. This was the first and most required part. As we will show the results, we achieved very successful results with fine tune. Due to its small size, the ball detection results were not like desired as much as player and referee detection. But we implemented basic Kalman Filter to overcome with it. To be honest, it was not also capable of detecting the exact ball position, but we obtained predicted ball positions instead of lack of ball detection with this filter. To sum up, we have got player, referee and ball bounding boxes with this model.

- **YOLO V8 Pose Estimation Model Introduction**

We obtained player and ball positions in video pixels, then what must be implemented? It is exactly homography transformation. Because we have to apply a transformation on location of video pixels into real football pitch metrics, we defined 32 points of pitch in our dataset. Our pose estimation model

predicted some important keypoint of football pitch and they became important reference points for our homography matrix.

- **Datasets Used in Our Project**

We have two different datasets as stated. One of them is for player, ball, referee detection and the other one is for pitch keypoints detection. We will provide dataset links below, but we can explain dataset labeling with examples from both datasets.

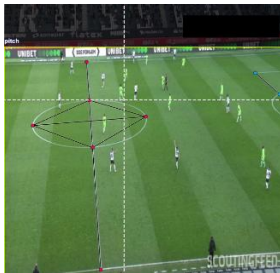


*Fig 1.0



*Fig 1.1

In the figure above, there are players, referees and a ball in addition to coaches, fans and other distracting objects. How can our model predict our three classes only? We used labeled datasets in the format of YOLO. We labeled players, referees and ball with bounding boxes. Each bounding box has top-left and bottom-center coordinates and confidence values. After labeling like 700 examples, we managed to get predictions with more confidence and lower loss value. We will provide graphs of fine-tune in the related part. These images are defined by Kaggle in usage of sport research. In Roboflow, you can discover several useful datasets for your fine-tune purposes. We obtain them from it. Labeling is illustrated in the example above, after labeling we have labels in the format of '.txt', each row represents class of bounding box, upper-left coordinate, lower-right coordinate like '2 0.21 0.37 0.021 0.053' in the image.



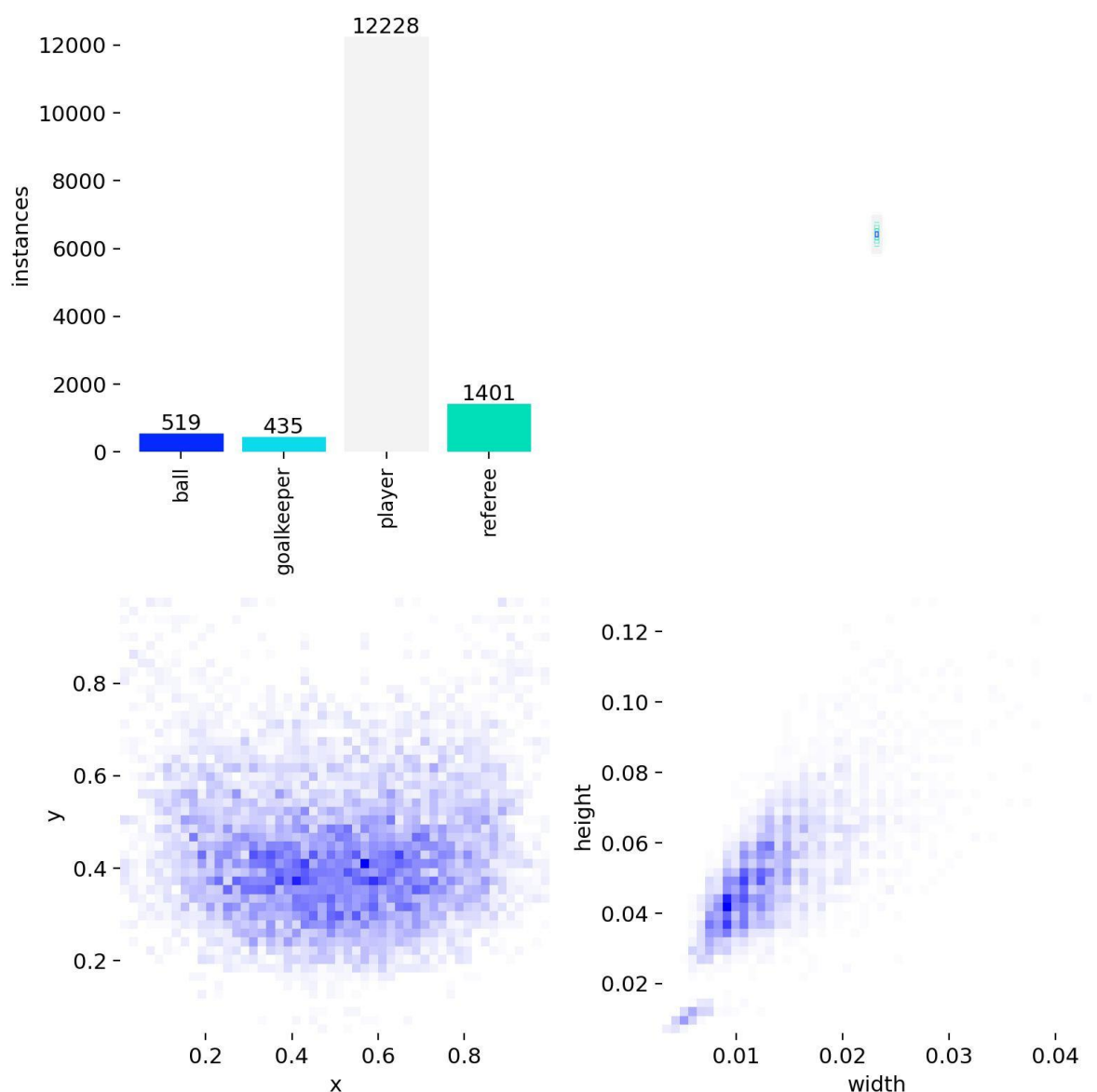
*Fig 1.3

Some of the keypoint labeling is illustrated in the example above. We have single class for pose estimation dataset and it is pitch. 32 keypoints represent different points of football pitch. We use broadcasting videos so we cannot acquire stable tracking. Camera is moving constantly and video pixel coordinates that detection model tells us are not stable. In addition to camera movement, we also have only a random part of pitch in a video. We cannot examine the video with panoramic and constant view, and all the pitch is not captured with broadcasting camera. So, we can apply a basic homography transformation to supply pitch representation coordinates. In the figure 1.3 defensive green-team players who are in a line are so close to penalty area. If we trusted the output of the detection model and didn't apply transformation, we would represent player next to goal because of

their positions which are at the end of the frame. Furthermore, they would not be represented in a line. Thanks to our keypoint detection model, we represent them at their exact position, near the penalty area and in a line.

Ball, Players, and Referees Detection Dataset: <https://universe.roboflow.com/roboflow-jvuqo/football-players-detection-3zvbc>

Pitch Keypoints Detection Dataset: <https://universe.roboflow.com/roboflow-jvuqo/football-field-detection-f07vi>



- **Related Work**

The field of object detection has undergone a significant transformation, moving from traditional hand-crafted feature extractors to robust deep learning architectures. This section reviews the foundational literature regarding the You Only Look Once (YOLO) framework and its specific applications in sports video analysis, particularly football.

- **Evolution of Real-Time Object Detection**

The paradigm of real-time object detection was revolutionized by Redmon et al. (2016) with the introduction of the original YOLO architecture. Unlike previous two-stage detectors such as Faster R-CNN, which utilized region proposal networks, YOLO framed object detection as a single regression problem. This approach allowed the network to predict bounding boxes and class probabilities directly from full images in a single evaluation. Although the initial version struggled with localization accuracy and small object detection, it established the foundation for high-speed computer vision systems that are now essential for live broadcast analysis.

Subsequent research has focused on balancing speed and accuracy through architectural refinements. As noted in a comprehensive review by Hussain (2023), the evolution from YOLOv1 to YOLOv8 has been characterized by the integration of Cross-Stage Partial Networks (CSPNet), improved feature fusion via Path Aggregation Networks (PANet), and the transition to anchor-free detection heads. These advancements have significantly reduced computational overhead while improving the mean Average Precision (mAP) on complex datasets like COCO and Roboflow.

- **Computer Vision in Football Analytics**

The application of these models to sports has become a major research trend. Shankara Narayanan et al. (2024) explored the use of YOLOv5 and YOLOv8 specifically for football data analytics. Their research highlights that while YOLO models are excellent for tracking players, detecting the football remains a challenge due to its small size and high velocity. The study demonstrated that YOLOv8 outperforms previous versions in extracting tactical insights such as ball possession and player movement patterns by utilizing its decoupled head and advanced data augmentation techniques.

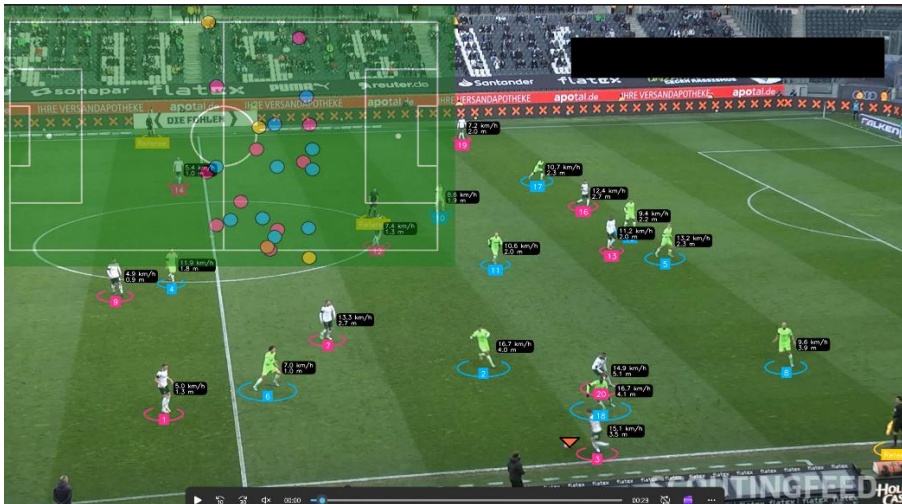
Furthermore, recent studies have emphasized the integration of tracking algorithms (such as ByteTrack or BoT-SORT) with YOLO detectors to maintain player identities across frames. This hybrid approach is now the standard for calculating physical metrics like distance covered and average speed without the need for wearable GPS sensors.

Resource Repositories and Differences Created by Our Projects

In this section, we will brief on what we have done distinctly in our project compared to referenced repositories and we will process differences in four topics but before distinctions, we must explain what we keep same with references. We used same model for object detection. One of the referenced repositories used YOLOv5m model but it is not considerable change made in our project. We collect some innovations from the secondary repository like homography transformation which is succeed by YOLOv8 pose estimation model. All the frame detections are collected and then all the data is processed, and video is annotated but it is not memory-safe method. We changed it with help of secondary repository. We detected everything with our models, and we annotated videos and processed data with detections. We didn't collect detections, yes, we did but in a manner of storing each detection of each frame so that we don't have to undergo video processing and can save time.

- **Homography Transformation**

Differently, again thanks to the secondary repository, we have managed to apply a transformation to the video pixels by some of the crucial points in the pitch and it was highly useful. We drawn excellent radar which represents pitch and the positions of ball, players and referees. In advance, we are now capable to get their exact pitch coordinates that are not affected by camera movements and view.



- **Ball Detection Consistency**

Initially, we must confess, YOLO model is not the best model for ball tracking and, as you will observe in the next section, in our dataset, there are few ball samples in contrast with the number of player and referee samples. All things considered, we have weak ball detection capabilities like although, ball is in the game, we cannot detect the ball due to occlusion and motion blur. In a temporary solution, we draw the imaginary ball position in a such case as occlusion, the ball is behind the player. In the first repository, for ball tracking, linear interpolation is used in case of undetected ball positions, but we applied different solution KALMAN FILTER. It was demonstrative work, but we think it provided more flexibility and precision. In the solution of linear interpolation, you must collect all the ball detections and create interpolation and fill the empty frames. It limits you to annotate frames at the end of last frame detection and it reduces performance. Moreover, you can create matrices for confidence of YOLO model detection and Kalman prediction.

- **Team Classification**

In the first repository, team classification is made by KMeans. For each player detection, you crop player bounding box and get a mean of shirt color in respect of RGB with KMeans. But it is not the best implementation. You leak some memory and it is not correct as much as solid transformer model SiglipVisionModel. We passed player crops to the model and got mean of the last dimension vectors over sequence length dimension (for example 224x224 image has 16x16 patches and sequence length is 14x14=196). Before averaging, dimension was (Batch Number, Patch Number, Hidden Dimension), our current dimension is (Batch Number, Hidden Dimension). Now our dimension

represents general features of our batch not of only single patch. Then we reduced them to 3 dimensions with UMAP and finally we created KMeans with the result. After collecting player shirt samples from across all frames and passing them to model to get KMeans, there is no preparation stuff, we can pass the crop and get the prediction.

- **Speed and Distance Estimation**

After collecting real pitch coordinates of players and ball, it is easy to estimate the speed of them. We averaged speed over like 8 frames and limited position difference between frames to rid of vibration in positions due to unclear detections. Maximum speed is defined as 30 km/h, 8.3 m/s, and our video was 25 fps. These are the values.

Reference Repositories

Repo 1: https://github.com/abdullahtarek/football_analysis

Repo 2: <https://github.com/roboflow/sports>

Models

In this section, we provide a detailed technical description of the neural network architectures implemented for the object detection task. Following the project requirements, we utilize a high-quality baseline repository and perform a comparative analysis between two different scales of the same architecture to evaluate the trade-offs between computational efficiency and predictive performance.

- **Baseline Model (Model 1): YOLOv8-Nano**

The baseline model selected for this project is YOLOv8-Nano (YOLOv8n), which represents the state-of-the-art in real-time object detection. The implementation is based on the official Ultralytics repository.

GitHub Repository: <https://github.com/ultralytics/ultralytics>

- **Architectural Overview**

YOLOv8 follows a "Single-Shot" detection paradigm, processing the entire image in a single forward pass. Its architecture is divided into three main components:

1. **Backbone (Feature Extractor):** The backbone utilizes a modified CSPDarknet53 structure. It is responsible for extracting hierarchical features from the input image. A key component here is the C2f module (Cross-Stage Partial Bottleneck with two convolutions), which enhances gradient flow and reduces computational bottlenecks. It also incorporates SPPF (Spatial Pyramid Pooling - Fast) at the final layer to extract features at various scales by pooling them into a fixed-size map.
2. **Neck (Feature Aggregator):** The neck uses a PANet (Path Aggregation Network) structure. It facilitates top-down and bottom-up information flow, ensuring that low-level spatial features and high-level semantic features are effectively fused. This is crucial for detecting objects of varying sizes.
3. **Head (Prediction):** YOLOv8 employs a Decoupled Head. Unlike previous versions where classification and localization were handled together, YOLOv8 separates these tasks into two independent branches. This separation allows the model to optimize the loss for each task more effectively.

- **Anchor-Free Detection**

One of the most significant architectural choices in YOLOv8 is the move to an Anchor-Free system. Instead of relying on predefined anchor boxes (which require manual tuning and can lead to sub-optimal results if the object shapes vary), the model directly predicts the center of an object and the distance to the box boundaries. This simplifies the network design and improves generalization across different datasets.

- **Model 2 (Comparison): YOLOv8-Medium**

For the comparison model, we selected Option B as defined in the project guidelines: the same architecture as Model 1 but with significant modifications in depth and width. We chose YOLOv8-Medium (YOLOv8m).

- **Justification for Choice**

While the Nano version is optimized for mobile and edge devices, the Medium version increases the number of channels and layers. This architectural scaling allows the model to learn more complex feature representations, potentially increasing the mean Average Precision (mAP) at the cost of higher computational complexity (GFLOPs).

- **Structural Differences**

Model 2 uses the same fundamental blocks (C2f, SPPF, PANet) but increases the "depth" (number of repeated blocks) and "width" (number of filters in the convolutions). This ensures a rigorous comparison by keeping the training methodology identical while only varying the model's capacity.

- **Training Scheme and Hyperparameters**

Both models were trained using a consistent scheme to ensure a fair comparison.

- **Loss Functions and Justification**

The total loss is a weighted sum of three distinct components, essential for the model to learn both "what" an object is and "where" it is:

1. **Classification Loss (Binary Cross Entropy - BCE):** Used in the classification branch to calculate the error between predicted class probabilities and the actual ground truth labels.
2. **Bounding Box Regression Loss (Complete IoU - CIoU):** This loss considers the overlap area, the distance between center points, and the aspect ratio. It provides a more stable and faster convergence compared to standard IoU.
3. **Distribution Focal Loss (DFL):** Used to refine the boundaries of the bounding boxes, especially in cases where the edges of an object are not clearly defined in the image.

- **Optimization and Backpropagation**

The optimization process utilizes **SGD (Stochastic Gradient Descent)** with a momentum of 0.937.

- **Backpropagation:**

During the training phase, the gradients of the total loss are computed with respect to every weight in the network using the chain rule. These gradients indicate the direction in which the weights should be adjusted to minimize the error. The optimizer then updates the weights based on the Learning Rate.

- **Hyperparameters:**

- **Initial Learning Rate:** 0.01
- **Epochs:** 10
- **Batch Size:** 16 (adjusted based on VRAM)
- **Image Size:** 640x640 pixels
- **Optimizer:** SGD with YOLOv8n Model and AdamW with YOLOV8m Model
- **Weight Decay:** 0.005

Experiments

This section details the training process and performance metrics for both implemented models. Both models were trained for 15 epochs using the same dataset splits to ensure a rigorous and fair comparison.

Model 1 Analysis: YOLOv8n (Baseline)

The baseline model, YOLOv8n, represents the smallest and fastest architecture in the YOLOv8 family, designed for high-speed inference.

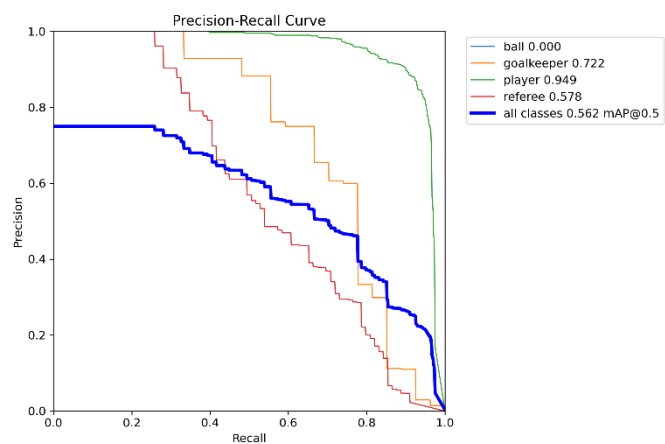
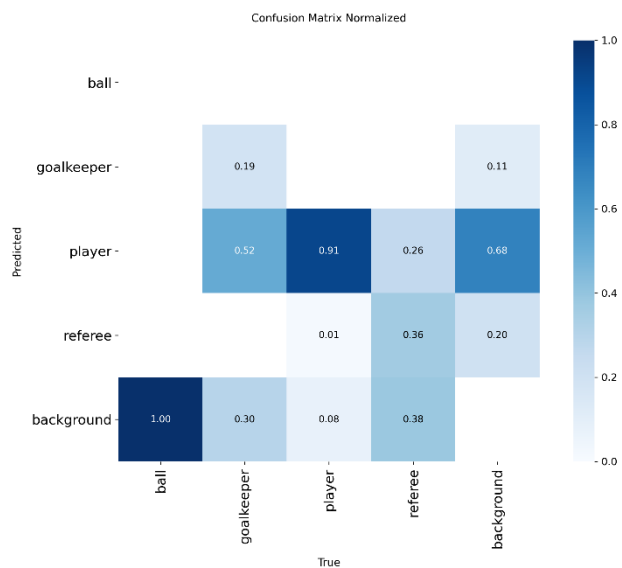
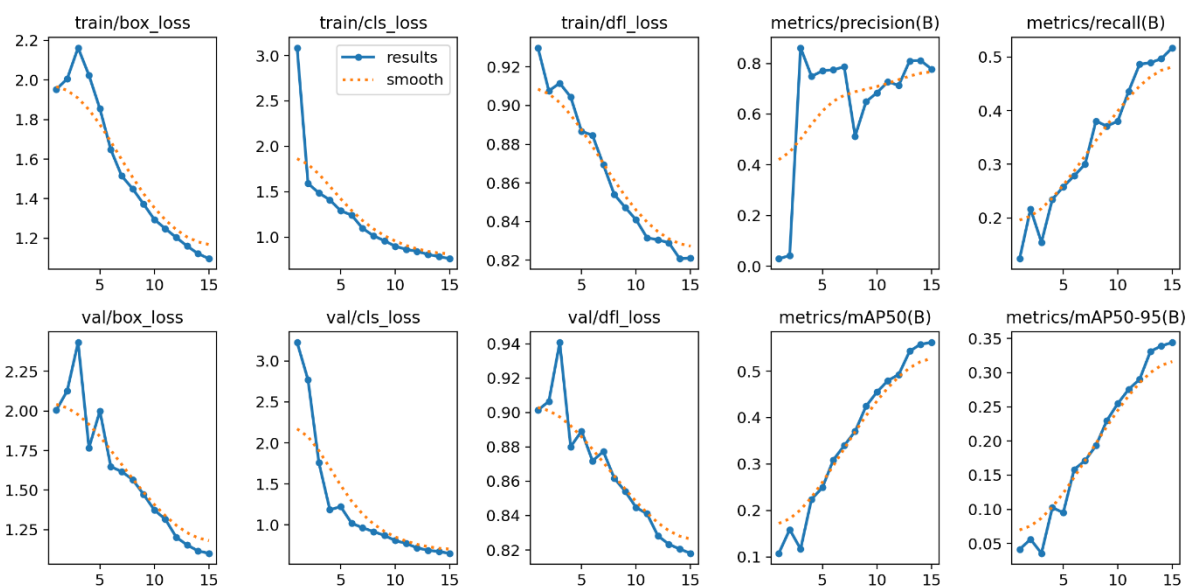
- **Training Dynamics:** According to the results.jpg, the training losses (box_loss, cls_loss, dfl_loss) showed a smooth and consistent decline⁴. The validation metrics closely followed the training trends, indicating a stable learning process without significant overfitting within 15 epochs.
- **Precision and Recall:** The BoxPR_curve.png shows that the model reached an overall mAP@0.5 of 0.562. The 'player' class achieved the highest precision (0.949), while the 'ball' class failed to register any successful detections (0.000).
- **F1-Score and Confidence:** The BoxF1_curve.png indicates an optimal F1-score of 0.52 at a 0.152 confidence threshold. This suggests the model is effective at identifying larger objects (players) even at low confidence levels, but struggles with class differentiation for smaller or rarer objects.
- **Confusion Matrix Analysis:** The normalized confusion matrix reveals that 100% of 'ball' instances were predicted as background. Additionally, 38% of 'referee' instances were misclassified as background, highlighting a weakness in detecting human figures that differ slightly from the standard 'player' profile.

Model 2 Analysis: YOLOv8m (Comparison)

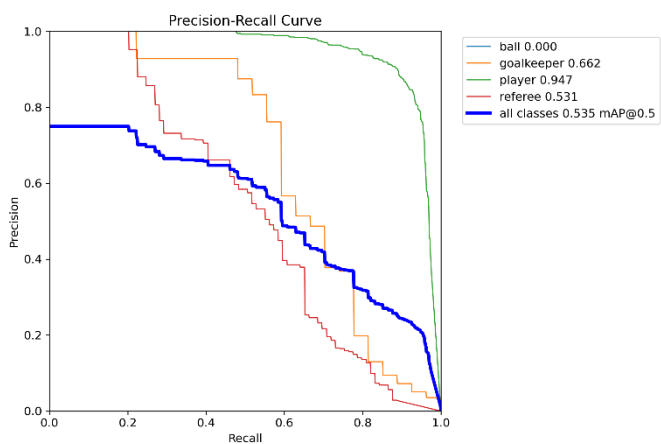
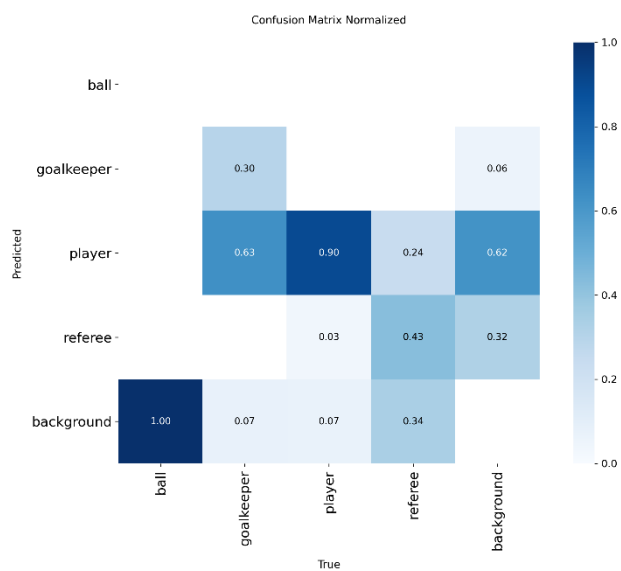
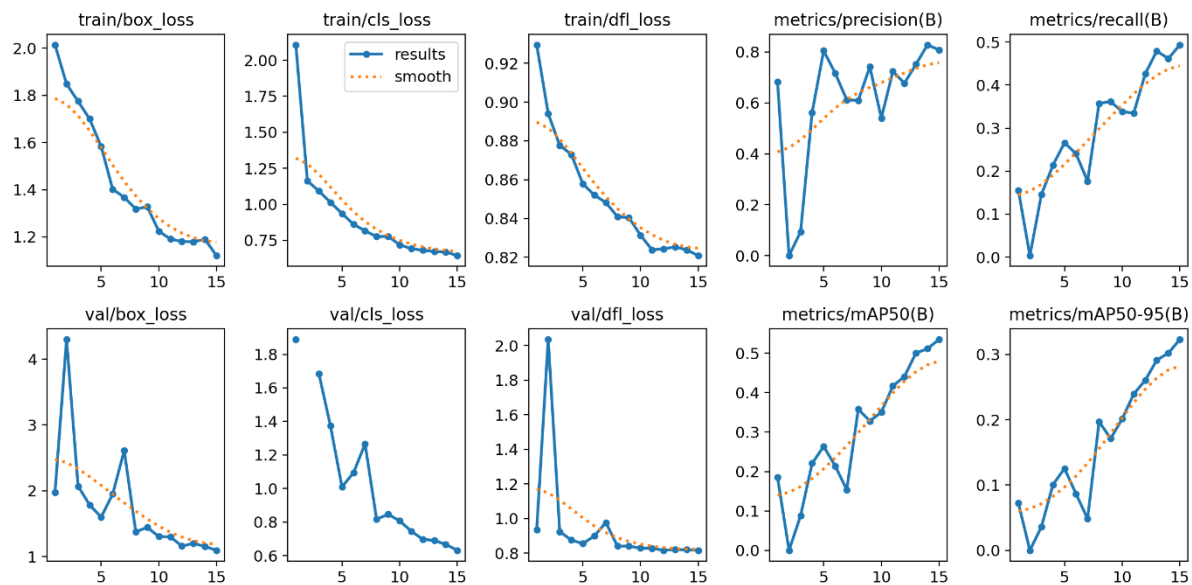
The second model, YOLOv8m, was chosen as a more complex architecture with a higher number of parameters to capture finer features.

- **Training Volatility:** Unlike the baseline, the medium model exhibited significant volatility during the early stages of training. The results.jpg for YOLOv8m shows a major spike in val/box_loss (exceeding 4.0) around the 2nd epoch before recovering. This suggests that the larger parameter space of the medium model requires a more stable initialization or a longer warm-up period.
- **Performance Metrics:** The model achieved an overall mAP@0.5 of 0.535, which is slightly lower than the Nano model. Class-specific results show 'player' at 0.947, 'goalkeeper' at 0.662, and 'referee' at 0.531.
- **Confidence Curves:** The BoxP_curve.png shows that while YOLOv8m can achieve a precision of 1.00, it requires a higher confidence threshold (0.830) compared to the Nano model to reach peak reliability.
- **Error Distribution:** The confusion matrix shows improved localization for the 'goalkeeper' class (only 7% false negatives as background compared to 30% in Nano), but it still suffered from a total failure in 'ball' detection (100% background error).

YOLOV8n



YOLOV8m



Comparison and Analysis

Systematic Performance Comparison

Comparing the two models reveals that the baseline (YOLOv8n) outperformed the more complex model (YOLOv8m) in overall mAP (0.562 vs 0.535) within the 15-epoch limit.

Metric	YOLOv8n (Model 1)	YOLOv8m (Model 2)
mAP@0.5 (All)	0.562	0.535
Player mAP	0.949	0.947
Goalkeeper mAP	0.722	0.662
Referee mAP	0.578	0.531
Ball mAP	0.000	0.000

Computational Complexity and Insights

- Training Efficiency: YOLOv8n demonstrated superior convergence speed. In short-term training (15 epochs), the smaller architecture reaches a stable state much faster than the medium architecture, which requires more iterations to optimize its larger weight matrix.
- Architectural Critique: Model 2 (YOLOv8m) has higher computational complexity. While it theoretically possesses a higher capacity for feature extraction, its performance was hindered by the limited training duration. The instability in its validation loss curves suggests it was still in the "exploration" phase of the loss landscape when training concluded.
- Small Object Detection Failure: The total failure of both models to detect the 'ball' is a critical limitation. This is attributed to the spatial resolution of the input images; the ball's pixel footprint is likely smaller than the network's final stride, causing it to be lost during downsampling in both architectures.

Conclusion

While YOLOv8m is a more powerful architecture, YOLOv8n is the better choice for this specific task given the constraints of the training scheme. For future improvements, increasing the number of epochs and implementing a specialized "small object" detection head or higher resolution input would be necessary to resolve the ball detection issue.